

Masterarbeit

Auf dem Weg zum autonom fahrenden Fahrrad: Eine videogestützte Regelung des Weges mit Webbots

Amine Karabila

Sommersemester 2025

Contents

1	Abstract	4
2	Einleitung	5
2.1	Problemstellung	5
2.2	Bisherige Arbeiten	5
2.2.1	Anna Ranz (2021) – Hardware-Grundlagen	5
2.2.2	Amine Karabila (2022) – Erste funktionierende Regelung	5
2.2.3	Jonah Zander (2023) – Optimierte PID-Balanceregung	6
2.2.4	Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung	7
3	Einleitung der angewandten Regelungstechnik	9
3.1	Fahrdynamische Modelle nach Schramm / Hiller / Bardini	9
3.1.1	Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik	9
3.1.2	Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik	9
3.1.3	Gültigkeitsbereiche und Modellvergleich	9
3.2	PID-Regelung – Prinzip und Eignung für das Balance-Problem	9
3.3	MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem	9
4	Webots-Simulationsumgebung	10
4.1	Überblick	10
4.2	Struktur der Simulationsumgebung	11
4.3	Das Fahrradmodell	13
4.3.1	Hinterrad (DEF rear_wheel)	13
4.4	Sensoren	14
4.5	Aktuatoren	16
4.6	Controller	16
4.7	Angebundene GUI	16
5	Kombinierte Regelung	16
5.1	Portierung des Balance-PID (Zander)	16
5.2	Portierung des Fahrtrichtungs-MPC (Giray)	16
5.3	Supervisions-/Koppelkaskade	16
5.4	Failsafe-Strategien	16

6	Ergebnisse	16
6.1	Teststrecken	16
6.1.1	Gerade Strecke mit Störimpuls	16
6.1.2	Slalom-Kurs	16
6.1.3	S-Kurve mit Steigung	16
6.1.4	Kreisfahrt	16
6.2	Balance-Stabilität	16
6.3	Spurtreue & Querabweichung	16
6.4	Rechenzeit & Echtzeitfähigkeit	16
7	Grenzen der Arbeit	17
8	Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)	17
9	Fazit	17
10	Literaturverzeichnis	17

1 Abstract

Diese Masterarbeit beschäftigt sich mit der Entwicklung einer kombinierten Regelungsstrategie für ein autonom fahrendes, selbstbalancierendes Fahrrad, basierend auf den Ergebnissen dreier vorangegangener Forschungsarbeiten. Ziel dieser Arbeit ist die erfolgreiche Integration einer PID-Balanceregulung mit einer MPC-basierten Fahrtrichtungsregelung, die mithilfe eines YOLO-Modells zur visuellen Spurdetektion ergänzt wird. Zur Validierung und Optimierung der kombinierten Regelung wurde die Simulationsumgebung Webots verwendet, wodurch die Robustheit und Funktionalität der entwickelten Regelungsalgorithmen realitätsnah überprüft werden konnten. Im Rahmen der Arbeit wurde eine zweistufige Regelungsarchitektur umgesetzt, welche die präzise Balancierung des Fahrrads mittels eines PID-Reglers und die zuverlässige Spurverfolgung sowie Navigation mithilfe eines MPC-Reglers sicherstellt. Die gewonnenen Simulationsergebnisse bestätigen die Machbarkeit und Effizienz des entwickelten Ansatzes und legen damit eine solide Grundlage für die zukünftige Implementierung der Regelung in einem realen autonomen Fahrrad.

2 Einleitung

2.1 Problemstellung

2.2 Bisherige Arbeiten

2.2.1 Anna Ranz (2021) – Hardware-Grundlagen

Die Bachelorarbeit von Anna Ranz bildete den Startpunkt des Projekts. Ihr Ziel war die Konstruktion eines lauffähigen Prototyps, mit dem grundlegende physikalische Eigenschaften eines selbstbalancierenden Fahrrads untersucht werden konnten. Kern der Hardware ist ein BeagleBone Black, der den Lenkmotor mittels MD49-Controller (EMG49) über UART und die Inertialmesseinheit BNO055 über I²C ansteuert. Ranz implementierte erste PID-Regelkreise, validierte die Sensorik in separaten Testprogrammen und analysierte die Dynamik des Gesamtsystems in Bezug auf Massenträgheit, Raddynamik und Lenkkinematik. Die umfangreiche Dokumentation enthält Schaltpläne, 3D-Modelle und Messaufnahmen und bildet damit die hardwareseitige Grundlage für alle späteren Arbeiten [12].

2.2.2 Amine Karabila (2022) – Erste funktionierende Regelung

Amine Karabila schloss 2022 unmittelbar an den Hardware-Prototyp von Ranz an und überführte das System von einzelnen Stellversuchen zu einem geschlossenen, echtzeitfähigen Regelkreis. Kernbestandteil ist eine dreistufige P-Regelung auf dem BeagleBone Black, die den Rollwinkel der BNO055-IMU [1] und die Lenkerstellung des MD49-Encoders [3] auswertet, während ein hinterer Nabenmotor die Vortriebsdrehzahl konstant hält [10]. Ein erstes Hindernis war die Strombegrenzung des MD49, die den EMG49-Lenkmotor zu langsam ansprechen ließ. Karabila ersetzte deshalb den Leistungsteil durch eine BTS7960-H-Brücke [8] und verwendete den MD49 nur noch zur hochauflösenden Wegmessung. Nicht-blockierende UART-Aufrufe verkürzten die Encoderabfrage von rund 4 ms auf knapp 1 ms, sodass die Gesamttransportzeit des Reglers auf etwa 2 ms sank. Die erste Regelversion koppelte die Motorgeschwindigkeit über einen gestreckten tanh-Verlauf an die Verweildauer des Rollwinkels in einer „habitablen Zone“. Außerhalb des Labors erwies sich dieses Verfahren als zu träge: Der Lenker schlug regelmäßig an die Endanschläge, und Stützräder mussten das Fahrrad vor dem Kippen bewahren. Die finale Lösung verknüpfte dagegen den gemessenen Rollwinkel direkt mit einem geschwindigkeitsabhängigen Soll-Encoderwert. Dadurch pendelte der Lenker nur noch mit etwa acht Ticks Amplitude um die Nullposition; ein integrierter Dämpfungsanteil reduzierte die Motordrehzahl automatisch, sobald sich das System seiner Soll-Lage näherte. Validiert wurde der Regler im Hof des Informatikgebäudes ($\approx 9.5 \times 26.5$ m, fünf Prozent Steigung). Bei Sollgeschwindigkeiten von 6–10 km/h blieb das Fahrrad während aller aussagekräftigen Fahrten aufrecht; Tip-Over-Events traten nicht auf. Die erfolgreichste Versuchsfahrt bei 10 km/h zeigte Spitzen von unter acht Encoder-Ticks und eine Querabweichung, die optisch kaum wahrnehmbar war. Abbildung 1 dokumentiert diese Fahrt und bietet sich als Platzhalter für Fotos der Versuchsstrecke an. Damit bewies Karabilas Arbeit erstmals die Realisierbarkeit einer selbstbalancierenden Regelung und schuf die Grundlage für die PID-Optimierungen von Zander sowie die MPC-Navigation von Giray.



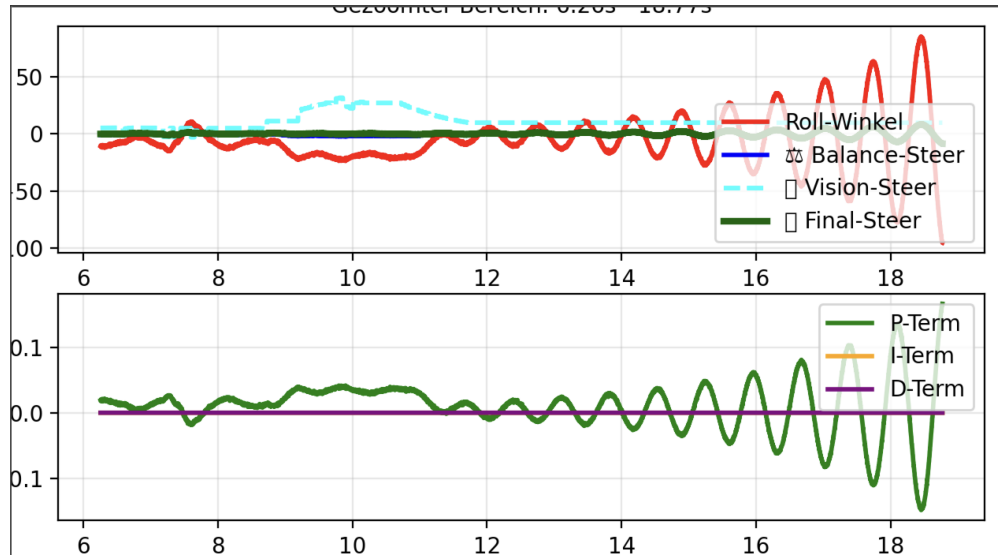


Figure 1: Erfolgreiche Versuchsfahrt von Karabila (2022) bei 10 km/h.

2.2.3 Jonah Zander (2023) – Optimierte PID-Balanceregung

Jonah Zander knüpfte 2023 an die vorherigen Arbeiten an und widmete sich der Verbesserung der Balanceregung des autonomen Fahrrads. Seine Bachelorarbeit „*Entwurf einer Steuerung für ein selbstbalancierendes Fahrrad*“ formulierte als Kernziel, dass das Fahrrad mindestens 60 Sekunden auf ebener Strecke selbstständig aufrecht fahren kann, ohne dass die Stützräder den Boden berühren [13]. Eingeleitet wurde diese Problemstellung mit dem Hinweis auf die allgemeinen Herausforderungen beim Entwurf eingebetteter Regelungssysteme – insbesondere das zuverlässige und rechtzeitige Auslesen verschiedener Sensoren (IMU, Encoder) und das Ansteuern der Aktoren (Lenk- und Antriebsmotor) unter Echtzeitbedingungen [13]. Auch alternative Ansätze wie die Balanceregung mittels Control-Moment-Gyroskopen wurden untersucht, beispielsweise in [11], jedoch mit deutlich komplexerer Hardware. Zander betonte hierbei die Komplexität, ein robustes Zusammenspiel dieser Komponenten sicherzustellen, und verwies auf die vorhandene Grundlage durch Ranz’ Hardware-Prototyp und Karabilas erste Regelungslösung [13]. Diese Vorarbeiten lieferten die Basis, auf der seine optimierte PID-Balanceregung aufbaute [10, 12].

Für die Umsetzung entwickelte Zander eine modular aufgebaute Software in C, die auf dem BeagleBone Black unter Debian-Linux als Multi-Thread-System läuft. Zeitkritische Aufgaben verlagerte er in die *Programmable Realtime Units* (PRUs) des Mikrocontrollers, um Sensorwerte ohne Verzögerung auszulesen. Insbesondere die Erfassung der Fahr- und Lenkwinkel wurde teils auf die PRUs ausgelagert, wodurch Lesezyklen im zweistelligen kHz-Bereich möglich wurden. Die Balance-Regelung selbst realisierte Zander mit einem klassisch ausgelegten PID-Regler, dessen Parameter auf Basis umfangreicher Tests und Logdaten schrittweise optimiert wurden. Damit verbesserte er die zuvor von Karabila genutzte dreistufige P-Regelung grundlegend, insbesondere durch Einführung eines Integral- und Dämpfungsanteils, um dauerhaftes aufrechtes Fahren zu ermöglichen. Ergänzend implementierte die Software umfangreiche Logging-Funktionen, mit denen Sensor- und Aktordaten während der Fahrt aufgezeichnet und anschließend grafisch ausgewertet werden konnten.

Neben der eigentlichen Regelungslogik verstärkte Zander auch die Betriebssicherheit des Gesamtsystems. So wurde die Hardware um mehrere Sicherheitsmechanismen erweitert: Zwei Notausschal-

ter ermöglichen das Abschalten von Lenkung bzw. Gesamtstrom, und eine *Reißleine* kappt beim Abreißen (z. B. durch den Eingriff einer betreuenden Person) sofort die Versorgung des Lenkermotors [13]. Dadurch wird verhindert, dass ein Softwarefehler zu unkontrolliertem Lenkereinschlag und mechanischer Beschädigung führt. Auch ein Fail-Safe für den Fernsteuer-Empfänger wurde integriert, der bei Signalverlust das Programm automatisch stoppt [13].

In experimentellen Fahrtests demonstrierte Zander schließlich den Erfolg seines Ansatzes. Das Fahrrad blieb über einen Zeitraum von mehr als 60 Sekunden stabil in Balance, ohne dass die Stützräder den Boden berührten [13]. Die Zielvorgabe einer mindestens einminütigen aufrechten Fahrt wurde damit erfüllt und sogar überschritten. Während der Testfahrt fuhr das System mit konstanter Geschwindigkeit auf ebenem Untergrund und bewältigte erstmals auch gesteuerte Richtungsänderungen: Über eingehende Signale einer Fernbedienung ließ sich das Fahrrad sicher nach links/rechts steuern, ohne die Balance zu verlieren [13]. Zander erzielte damit den bis dahin längsten autonom balancierten Fahrbetrieb des Prototyps und schuf die Grundlage, auf der weiterführende Regelungsansätze aufsetzen können [13]. Seine Arbeit markiert somit einen wichtigen Meilenstein – mit einer ausgereiften Balance-Regelung, die eine zuverlässige Plattform für höhere Regelungsschichten bietet. Entsprechend wird die von Zander entwickelte PID-Regelung in der vorliegenden Masterarbeit unmittelbar weiterverwendet: Sie wurde für die Webots-Simulation portiert und bildet als unterste Ebene des zweistufigen Reglers ein zentrales Fundament des Gesamtsystems. 

2.2.4 Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung

Nach der Entwicklung der grundlegenden Balance-Steuerung in der Bachelorarbeit von Zander [13], die das selbstbalancierende Fahrrad erstmals über mindestens 60s autonom stabil fahren ließ, richtete sich der Fokus der folgenden Arbeiten auf die Erweiterung um autonome Navigationsfähigkeiten. Mehrere Vorarbeiten hatten bereits die Hardware-Plattform und Basissoftware geschaffen [10,12,13]. Darauf aufbauend adressierte die Masterarbeit von Ghidai [4] die noch offene Problemstellung, das Fahrrad mithilfe einer kamerabasierten Umgebungserfassung und geeigneter Regelung autonom fahren zu lassen.

Problemstellung. Das selbstbalancierende Fahrrad verfügte zwar über Aktuatoren (Lenkmotor, Antrieb) und Sensoren (etwa IMU und Encoder) sowie eine funktionierende Balance-Regelung [13], jedoch fehlten bis dato eine visuelle Orientierung im Raum und eine automatische Steuerungslogik für die Fahrtrichtung. Ghidai formulierte daher als Ziel, eine *Computer-Vision*-Pipeline auf einem eingebetteten System zu integrieren, welche dem Fahrrad eine Wahrnehmung der Umgebung und Trajektorienplanung ermöglicht [4]. Insbesondere sollten Objekte bzw. Fahrbahnelemente in der Kamerasicht erkannt und daraus eine Soll-Fahrspur abgeleitet werden. Gleichzeitig war ein Regler zu entwickeln, der das Fahrrad entlang dieser vorgegebenen Trajektorie steuert, ohne die aufrechte Balance zu verlieren. Diese Aufgabenstellung ist anspruchsvoll, da die zuverlässige Objekterkennung in Echtzeit auf eingebetteter Hardware limitierte Rechenressourcen erfordert und die Regelung eines selbstbalancierenden, nichtlinear instabilen Systems komplex ist.

Vorgehen und zentrale Beiträge. Ghidais Lösungsansatz besteht aus einer zweistufig verarbeiteten Pipeline aus Bildverarbeitung und Regelungstechnik. Als erster Schritt wird ein KI-basiertes Verfahren zur Objekterkennung im Videobild eingesetzt. Hier evaluierte Ghidai verschiedene aktuelle Modelle der *Computer Vision*, unter anderem das “*Segment Anything*”-Modell FastSAM [14] und die YOLOv8-Architektur [9], hinsichtlich ihrer Eignung für den eingebetteten Einsatz. Aufgrund seiner hohen Erkennungsgenauigkeit und Effizienz fiel die Wahl auf ein *Ultralytics YOLOv8*-Netzwerk in der *Segmentation*-Variante [9]. Dieses Convolutional Neural Net-

work wurde mit einem angepassten Datensatz trainiert, um relevante Objekte der Fahrumgebung (wie z. B. Fahrbahnbegrenzungen oder Hindernisse) zu segmentieren. Die Inferenz konnte durch Optimierung mit NVIDIA *TensorRT* auf dem eingesetzten NVIDIA Jetson Nano Board beschleunigt werden, sodass die Bildauswertung in Echtzeit (Zyklusdauer ≈ 100 ms) erfolgt [4]. Anhand der segmentierten Bilddaten werden anschließend durch Extraktion von Stützpunkten und Spline-Interpolation kontinuierliche Referenztrajektorien berechnet, welche die gewünschte Fahrlinie des Fahrrads repräsentieren [4].

Als zweiter Schritt implementierte Ghidai einen geeigneten Fahrregler, um der berechneten Trajektorie zu folgen. Hierbei kam ein *Model Predictive Controller* (MPC) zum Einsatz, der auf einem Einspur-Fahrdynamikmodell des Fahrrads basiert [7]. Ein MPC optimiert iterativ die Stellgrößen (Lenkwinkel, ggf. Beschleunigung) über einen endlichen Zeithorizont, um Abweichungen von der Soll-Trajektorie zu minimieren, und hat sich in der Fahrzeugführung als effektiver Ansatz bewährt [7]. In Ghidais Architektur wird der MPC in jedem Zyklus mit der aktuellen Zustandsschätzung gefüttert und liefert Lenkbefehle, die das Fahrrad entlang der Referenzspur halten [4]. Da für die Jetson-Plattform keine MATLAB-basierte MPC-Toolbox verfügbar war, wurden das Einspurmodell, die Trajektorien-Interpolation und zunächst auch der Regler in Python implementiert, bevor ein Open-Source-MPC-Modul integriert werden konnte [4]. Die gesamte Pipeline – von der Kamerabild-Auswertung bis zur Berechnung der Stellgrößen – läuft dabei auf dem Jetson Nano und erfüllte die Echtzeitanforderungen des Systems (Verarbeitung alle 100 ms) [4].

Ergebnisse und Ausblick. Die Masterarbeit demonstrierte, dass eine monokamera-basierte autonome Steuerung des selbstbalancierenden Fahrrads prinzipiell umsetzbar ist. Das entwickelte System erkannte Fahrbahnkanten und einfache Hindernisse und konnte in Simulationen eine vorgegebene Strecke abfahren [4]. Gleichzeitig zeigten sich Grenzen und offene Herausforderungen, die einen Ausblick auf weitere Forschungsschritte geben. So war die implementierte Pipeline zwar in der Lage, Hindernisse zu detektieren, besaß jedoch noch keine Strategie zu deren Umfahrung – zukünftige Arbeiten könnten hier ansetzen, indem sie die Trajektorienplanung um kollisionsvermeidende Manöver erweitern. Des Weiteren ließen sich die Umgebungswahrnehmung und Regelungspräzision durch Tiefeninformationen verbessern. Ghidai schlägt vor, entweder einen stereoskopischen Kamerasensor zu integrieren oder mittels Monokamera-Methoden einen Tiefengradienten zu ermitteln, um die Distanz zu erkannten Objekten abzuschätzen [4]. Solche Daten könnten genutzt werden, um die berechnete Referenzkurve für den MPC-Regler mit Entfernungsinformationen zu gewichten und damit die Regelgüte zu erhöhen. Auch aus technischer Sicht sind Optimierungen möglich: Neuere KI-Modelle und Hardware-Plattformen versprechen höhere Performance. Beispielsweise erschienen 2024 beschleunigte Segmentierungsansätze, die eine um den Faktor 6 gesteigerte Verarbeitungsgeschwindigkeit bieten, sowie weiterentwickelte YOLO-Modelle [9, 14]. Leistungsfähigere Edge-AI-Hardware (etwa der aktuelle Raspberry Pi 5 mit KI-Beschleunigung) könnte die Echtzeitfähigkeit weiter verbessern. Schließlich wurde in Ghidais Arbeit die kamerabedingte Beeinträchtigung durch Neigungen des Fahrrads (Rotation um die Rollachse) nicht gelöst; hier wird als hardwareseitige Maßnahme ein *Gimbal* zur Stabilisierung der Kamera während der Fahrt vorgeschlagen [4].

Einordnung für die aktuelle Arbeit. Die Regelungsansätze aus den Arbeiten von Zander und Ghidai werden in der vorliegenden Masterarbeit zusammengeführt. Konkret wird die von Zander entwickelte Balanceregung (innere Schleife) mit dem von Ghidai entworfenen MPC-basierten Spurhalte-Regler (äußere Schleife) in einer zweistufigen Simulationsarchitektur kombiniert. Diese Integration ermöglicht es, das Zusammenwirken von Balancehaltung und autonomen Lenkmanövern zunächst virtuell zu erproben und abzustimmen, bevor es in realen Fahrversuchen umgesetzt wird. Damit knüpft die aktuelle Arbeit direkt an den Stand der Technik an und erweitert ihn um die

gleichzeitige Betrachtung beider Regelungsebenen für das selbstbalancierende Fahrrad.

3 Einleitung der angewandten Regelungstechnik

3.1 Fahrdynamische Modelle nach Schramm / Hiller / Bardini

3.1.1 Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik

3.1.2 Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik

3.1.3 Gültigkeitsbereiche und Modellvergleich

3.2 PID-Regelung – Prinzip und Eignung für das Balance-Problem

3.3 MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem

4 Webots-Simulationsumgebung

4.1 Überblick

Webots ist ein frei verfügbares, plattformunabhängiges Robotiksimulationsprogramm, das seit 1998 von Cyberbotics kontinuierlich weiterentwickelt wird. Es kombiniert eine moderne Qt-Oberfläche mit dem auf ODE basierenden Physik-Kernel und einer OpenGL-basierten Rendering-Engine und ermöglicht so fotorealistische 3D-Szenarien inklusive deterministischer Dynamiksimulation [2]. Die Software stellt eine hierarchische Szenenstruktur bereit: *World-Dateien* (*.wbt) enthalten Knoten für Roboter, Objekte und Umweltparameter; jeder Roboter wiederum kapselt Sensor- und Aktuatorknoten, die über Controller-Prozesse in C/C++, Python, Java, MATLAB oder via ROS-Bridge angesteuert werden können [5]. Ein spezieller *Supervisor-API* erlaubt Eingriffe auf Welt-Ebene (z. B. Reset, Logging, Auto-Evaluation) und bildet damit die Grundlage für reproduzierbare Experimente und Wettbewerbe. Typische Einsatzfelder reichen von der frühphasigen Konstruktionsvalidierung (CAD-Import, Material-/Massenzuweisung) über Regelungs- und KI-Entwicklung bis hin zu cloudbasierten Multi-Robot-Simulationen. Studien zeigen zudem, dass Webots dank der offenen Schnittstellen zunehmend als Digital-Twin-Plattform für neuartige Robotertypen – etwa weiche Greifer – genutzt wird, ohne dass Spezialhardware erforderlich ist [6].



Figure 2: Übersicht der Webots-Simulationsumgebung.

4.2 Struktur der Simulationsumgebung

Die verwendete Simulationsumgebung basiert auf Webots und damit auf einer hierarchisch organisierten Szenenbeschreibung im VRML-Format. Eine `.wbt`-Datei kapselt sämtliche Entitäten der virtuellen Welt – von globalen Parametern (Zeitauflösung, Kontaktmodelle) über Umgebungsobjekte (Boden, Hintergrund, Lichtquellen) bis hin zu Robotern mitsamt deren Sensorik, Aktuatorik und Controllern [2, 5]. Jeder Knoten (Node) kann wiederum Unterknoten besitzen, sodass sich komplexe Systeme modular aus standardisierten Bausteinen zusammensetzen lassen. Für die vorliegende Arbeit ist diese Hierarchie entscheidend, weil sie erlaubt, (i) das physikalische Verhalten des Fahrrads getrennt von (ii) der Szenengeometrie und (iii) den übergeordneten Reglerprozessen zu modellieren. Änderungen an einem Teilbereich – etwa an den Reibungsparametern der Reifen oder an der Position einer Kamera – bleiben somit lokal und beeinträchtigen die übrigen Komponenten nicht.

Ausgangspunkt der Welt ist eine rechteckige Bodenarena mit texturierter Fahrbahnoberfläche, die als Referenzstrecke für die visuelle Spurführung dient. Globale Kontakt- und Reibungsparameter sind hier definiert, um das Zusammenspiel zwischen Reifen und Untergrund konsistent zu halten. Ein fester Zeitschritt im Millisekundenbereich stellt sicher, dass sowohl die Balancebewegungen des Fahrrads als auch die Bildverarbeitungsschleifen deterministisch und in Echtzeit simuliert werden können [2]. Die Kameraperspektive des Betrachters (Viewpoint) folgt dem Fahrrad aus erhöhter Position, was die visuelle Analyse der Fahrmanöver während der Versuche erleichtert.

Das zentrale Element der Szene ist der Roboterknoten *BICYCLE*, der das selbstbalancierende Fahrrad repräsentiert. Sein Geometriemodell wurde aus CAD-Daten übernommen und mit vereinfachten Kollisionsvolumina (Boxen und Zylinder) hinterlegt, um die Physiks simulation effizient zu halten. Die Massen- und Trägheitsparameter der Baugruppen (Rahmen, Räder, Gabel) sind explizit angegeben, sodass die Dynamik – insbesondere Roll- und Nickbewegungen – realitätsnah nachgebildet wird. Bewegliche Komponenten sind über Hinge-Joints angebunden: Das Hinterrad wird über einen Rotationsmotor für den Vortrieb angesteuert, die Lenkbewegung des Vorderrads erfolgt über ein zweites Gelenk mit eigenem Servomotor. Ergänzend existiert ein Kurbel-/Pedal-Gelenk, das in erster Linie der geometrischen Vollständigkeit und nicht der Antriebserzeugung dient. Zur Zustandserfassung sind Sensorschnittstellen integriert: Eine Inertialmesseinheit (IMU) liefert dem Balance-Regler fortlaufend Orientierung und Rollwinkel [1]; eine am Fahrrad montierte Kamera kann genutzt werden, um on-board visuelle Informationen auszuwerten. Ein Display-Modul steht optional für Debug- oder Visualisierungszwecke zur Verfügung. Die Kommunikation zwischen Fahrrad und übergeordnetem Regler erfolgt über ein Paar aus **Emitter** und **Receiver**, die einen leichten, paketbasierten Nachrichtenaustausch innerhalb von Webots ermöglichen.

Übergeordnet ist ein *Supervisor*-Knoten angelegt. Dieser spezielle Robotertyp besitzt in Webots weitreichende Eingriffsmöglichkeiten auf Welt-Ebene (z. B. Position-Reset, Logging, automatisierte Auswertung) und dient hier als externe Beobachter- und Steuerinstanz. Der Supervisor trägt eine statisch in der Szene platzierte Kamera, die die Fahrbahn aus einer globalen Perspektive erfasst. Auf Basis dieser Bilder führt der Python-basierte Vision-Controller eine Spurdetektion (YOLO) durch und berechnet über ein modellprädiktives Regelungsverfahren (MPC) Lenksollwerte bzw. Trajektorien. Diese Kommandos werden per Funkkanal (Emitter) an den Balance-Controller des Fahrrads übertragen, der sie in konkrete Motorbefehle umsetzt. Umgekehrt sendet das Fahrrad Statusdaten zurück, die der Supervisor für Evaluations- und Loggingzwecke speichert. Dieses Zusammenspiel zweier Controller-Prozesse – ein echtzeitnaher PID-Regler in C auf Roboterseite und ein bildverarbeitungsgetriebener MPC-Regler in Python auf Supervisorebene – bildet die Kernstruktur der kombinierten Regelung.

Die gewählte Struktur der Simulationsumgebung verfolgt zwei Ziele: Erstens ermöglicht sie reproduzierbare und sichere Experimente in verschiedensten Fahrszenarien (Geradeausfahrt mit Störimpuls, Slalomkurse, Kurvenfahrten mit Steigung), ohne das reale Hardware-Risiko des Umkippen oder der Beschädigung einzugehen. Zweitens gestattet die klare Trennung von physikalischem Modell, Sensorebene und Reglerlogik eine gezielte Variation einzelner Parameter (z. B. Reibung, Zeitschritt, Bildauflösung), um deren Einfluss auf die Gesamtperformance systematisch zu untersuchen. Webots fungiert damit als flexibles Testbett, in dem sowohl Balance- als auch Spurführungsalgorithmen iterativ entwickelt, validiert und optimiert werden können, bevor sie auf den realen Prototyp übertragen werden.

4.3 Das Fahrradmodell

Das folgende Kapitel führt in das in Webots implementierte Fahrradmodell ein, wie es in der `.wbt`-Datei spezifiziert ist. Das Modell ist als kinematischer Baum aus starren Körpern (`Solid`) aufgebaut, die über eindimensionale Drehgelenke (`HingeJoint`) verbunden sind. Die sichtbare Geometrie (`CadShape`) ist vom physikrelevanten Kollisionsmodell (`boundingObject` mit einfachen Primitiven) getrennt; Masse, Schwerpunkt und Trägheit sind im `Physics`-Block definiert. An ausgewählten Gelenken wirken `RotationalMotor`-Aktuatoren, während Dämpfung und Haftreibung über `HingeJointParameters` festgelegt sind. In den nachfolgenden Unterabschnitten werden die Baugruppen des Fahrrads entlang der Knotenstruktur der Weltdatei beschrieben und deren Parameter hinsichtlich ihres Einflusses auf die Dynamik (Lenkung, Rad-Boden-Kontakt, Eigendämpfung) erläutert.



Figure 3: Das Fahrradmodell in Webots.

4.3.1 Hinterrad (`DEF rear_wheel`)

Der Knoten `rear_wheel` ist Teil des Gesamtfahrrads und definiert das angetriebene Hinterrad (`HingeJoint`). Die `HingeJointParameters` beschreiben das Lager des Hinterrads. Der `anchor` `0 0.5312 0` gibt die Position der Radachse im Elternkoordinatensystem an. Die `dampingConstant` `0.8` ist die viskose Dämpfung des Lagers: höher bedeutet mehr Beruhigung gegen Schwingungen, aber eine trägere Rotation. Die `staticFriction` `0.1` ist die Haftreibung im Lager: sie verhindert kleines Flattern nahe Stillstand, erhöht jedoch das zum Anfahren nötige Drehmoment. Das `endPoint Solid` ist der vom Gelenk bewegte starre Körper auf der Kind-Seite des `HingeJoint`. Er rotiert um den am Gelenk definierten Anker/Achse und bildet damit das eigentliche Bauteil (hier: das Hinterrad), das durch die Bewegung betroffen ist. In der Hierarchie gilt: Elternteil $\rightarrow$ Gelenk $\rightarrow$ `endPoint Solid`; alles, was hier angehängt ist, bewegt sich mit. Das `boundingObject` definiert die kollisions- und kontaktrelevante Geometrie des Hinterrads, unabhängig vom sichtbaren Mesh. Es bestimmt, wo das Rad den Boden „berührt“, wo Kräfte eingeleitet werden und wie Kollisionen erkannt werden. Im Modell ist es ein einfacher Zylinder, der per `Pose`-Rotation (entspricht Drehung um die y -Achse um 90°) in die Radebene gedreht wird; so bildet er die Kontaktfläche des Reifens ab. Zusammen mit dem `contactMaterial` (z. B. `"wheel"`) entstehen daraus die Reib- und Rückpralleigenschaften im Kontakt mit dem Untergrund. Größe und Ausrichtung des `boundingObject` sollten zur realen Radgeometrie passen, da es die Kontaktlage und damit das Fahrverhalten maßgeblich beeinflusst.

Physik (Physics) Der `Physics`-Block definiert die Masseneigenschaften des Hinterrads. `density` `-1` bedeutet: Masse und Trägheit werden *nicht* aus dem `boundingObject` berechnet, sondern mit `mass`, `centerOfMass` und `inertiaMatrix` explizit vorgegeben. `mass` `2` setzt die Gesamtmasse (kg), `centerOfMass` `[0 0 0]` positioniert den Schwerpunkt im `Solid`-Frame (hier an der Nabe), und `inertiaMatrix` `[0.122 0.122 0.245 0 0 0]` legt die Rotations-Trägheiten (kg·m²) fest; größere Werte bedeuten trägere Reaktion auf Dreh- bzw. Kippbewegungen. Die Off-Diagonale `0 0 0` zeigt

an, dass Hauptachsen und Solid-Achsen ausgerichtet sind. Für automatische Berechnung könnte statt `density -1` eine positive Dichte verwendet werden (dann kommen Masse/Trägheit aus Volumen×Dichte des `boundingObject`).

Listing 1: Hinterrad—Kinematik, Kontakt und Physik (Auszug)

```

1  DEF rear_wheel HingeJoint {
2    jointParameters HingeJointParameters {
3      anchor 0 0.5312 0
4      dampingConstant 0.1
5      staticFriction 0.8
6    }
7    device [...] # RotationalMotor (siehe Kapitel 4.5 Aktuatoren)
8    endPoint Solid {
9      children [
10       Transform {...} # visuelles Mesh (ohne Physik)
11     ]
12     contactMaterial "wheel"
13     boundingObject Pose {
14       rotation 0 1 0 1.5708
15       children [ Cylinder { height 0.05 radius 0.352 subdivision 72 } ]
16     }
17     physics Physics {
18       density -1, mass 2, centerOfMass [ 0 0 0 ],
19       inertiaMatrix [ 0.122 0.122 0.245 0 0 0 ]
20     }
21   }
22 }
23

```

4.4 Sensoren

Das System nutzt zwei primäre Sensorquellen:

- **Inertialmesseinheit (IMU, Gerät „imu“)** am Fahrrad für die Rollwinkelmessung.
- **Kamera (Gerät „camera“)** am Vision-Controller zur Spur-/Straßenerkennung.

Die IMU treibt die innere, schnelle Stabilisierung (Balance), die Kamera die äußere, langsamere Pfadführung (Vision). Beide werden in einer einfachen, statischen Fusion zusammengeführt.

IMU: Implementierung (Webots & Code)

Konfiguration in der .wbt-Datei Kamera: Vision-basierte Spurführung Die Kamera liefert Bildframes, die zur *Spur-/Straßenerkennung* verarbeitet werden:

1. **Merkmalsextraktion:** YOLO-Segmentierung (falls verfügbar) oder ein Fallback (Kanten/ROI) bestimmt den lateralen Fehler e relativ zur Bildmitte.
2. **Vision-PID:** Ein separater PID-Regler bildet daraus einen normierten Lenkbefehl $u_{\text{vis}} \in [-1, 1]$ sowie eine Geschwindigkeits-*Empfehlung* $u_v \in [0, 1]$ (bei großem Fehler wird Geschwindigkeit reduziert).

3. **Zeitbasis:** Die Verarbeitung läuft bewusst langsamer (≈ 20 Hz) als die Balance-Regelung.

Sensordatenfusion und Einbindung in die Gesamtregelung Die Fusion erfolgt im Balance-Controller:

$$\theta_{\text{vis}} = u_{\text{vis}} \cdot \theta_{\text{max}}, \quad (1)$$

$$\theta_{\text{final}} = w_v \cdot (-\theta_{\text{vis}}) + w_b \cdot \theta_{\text{bal}}, \quad w_v, w_b \in [0, 1], \quad w_v + w_b = 1, \quad (2)$$

$$v_{\text{target}} = v_{\text{min}} + u_v (v_{\text{max}} - v_{\text{min}}). \quad (3)$$

Dabei sind w_v und w_b konfigurierbare Gewichte. θ_{final} und v_{target} werden jeweils auf zulässige Bereiche begrenzt. Bei Vision-Timeout bzw. ungültigem Kommando läuft ausschließlich die Balance-Regelung ($\theta_{\text{final}} = \theta_{\text{bal}}$, $v_{\text{target}} = v_{\text{base}}$).

Kommunikation und Status Der Vision-Controller sendet periodisch $\{u_{\text{vis}}, u_v\}$ sowie Diagnosewerte (z. B. Fehler, PID-Terme, Maskenabdeckung) an den Balance-Controller. Umgekehrt publiziert der Balance-Controller an den Vision-Controller u. a. den aktuellen Rollwinkel, den angewandten Lenkwinkel und einen Stabilitätsindikator. Die Statusdaten dienen Visualisierung/Debug und fließen nicht in die Balance-PID-Schleife ein.

4.5 Aktuatoren

4.6 Controller

4.7 Angebundene GUI

5 Kombinierte Regelung

5.1 Portierung des Balance-PID (Zander)

5.2 Portierung des Fahrtrichtungs-MPC (Giray)

5.3 Supervisions-/Koppelkaskade

5.4 Failsafe-Strategien

6 Ergebnisse

6.1 Teststrecken

6.1.1 Gerade Strecke mit Störimpuls

6.1.2 Slalom-Kurs

6.1.3 S-Kurve mit Steigung

6.1.4 Kreisfahrt

6.2 Balance-Stabilität

6.3 Spurtreue & Querabweichung

6.4 Rechenzeit & Echtzeitfähigkeit

- 7 Grenzen der Arbeit
- 8 Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)
- 9 Fazit
- 10 Literaturverzeichnis

References

- [1] Bosch Sensortec. Bno055 intelligent 9-axis absolute orientation sensor. <https://www.bosch-sensortec.com>, 2020.
- [2] Cyberbotics. Webots – open source robot simulator. <https://cyberbotics.com>, 2025.
- [3] Devantech Ltd. Md49 dual motor driver datasheet. <https://www.robot-electronics.co.uk>, 2019.
- [4] Yassin Ghidaei. Video- und sensorgestütztes autonomes fahren für ein selbstbalancierendes fahrrad. Master’s thesis, Goethe-Universität Frankfurt am Main, 2024.
- [5] R. Gupta and S. Lee. Evaluating webots as digital-twin platform for multi-robot systems. In *Proc. IEEE ICRA*, 2023.
- [6] M. et al. Hadi. Soft-gripper simulation in webots. *Soft Robotics*, 11(2):123–135, 2024.
- [7] Alex Hunziker. Autonomous steering of an electric bicycle based on sensor fusion using model predictive control. Master’s thesis, Goethe-Universität Frankfurt am Main, 2019.
- [8] Infineon Technologies. Bts7960 high-current half-bridge driver. <https://www.infineon.com>, 2018.
- [9] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Yolo by ultralytics. <https://github.com/ultralytics/ultralytics>, 2023.
- [10] Amine Karabila. Auf dem weg zum selbstfahrenden fahrrad: Analyse und implementierung einer regelung der balance. Master’s thesis, Goethe-Universität Frankfurt, 2022.
- [11] Sang-Hyun Park and Seung-Yun Yi. Active Balancing Control for Unmanned Bicycle Using Scissored-pair Control Moment Gyroscope. *International Journal of Control, Automation and Systems*, 18(1):217–224, 2020.
- [12] Anna Ranz. Entwicklung eines selbstbalancierenden fahrradprototyps. Master’s thesis, HWR Berlin, 2021.
- [13] Jonah Zander. Entwurf einer Steuerung für ein selbstbalancierendes Fahrrad. Bachelorarbeit, Frankfurt University of Applied Sciences, 2023.
- [14] Xu Zhao, Wenchao Ding, Yongqi An, Yinglong Du, Tao Yu, Min Li, Ming Tang, and Jinqiao Wang. Fast segment anything. *arXiv:2306.12156 [cs.CV]*, 2023.